SERVLETS

- **Servlet is a technology used to create a web application.**
- **A Servlet is a Java class used to build server-side web applications.**
  - **It runs inside a web server / application server (like Tomcat)**
  - **It handles client requests and generates dynamic responses.**

Why do we need Servlets?

- **Before servlets:**
- **Web pages were static (HTML only)**
- **No database interaction**
- **Same response for all users**

Servlets allow:

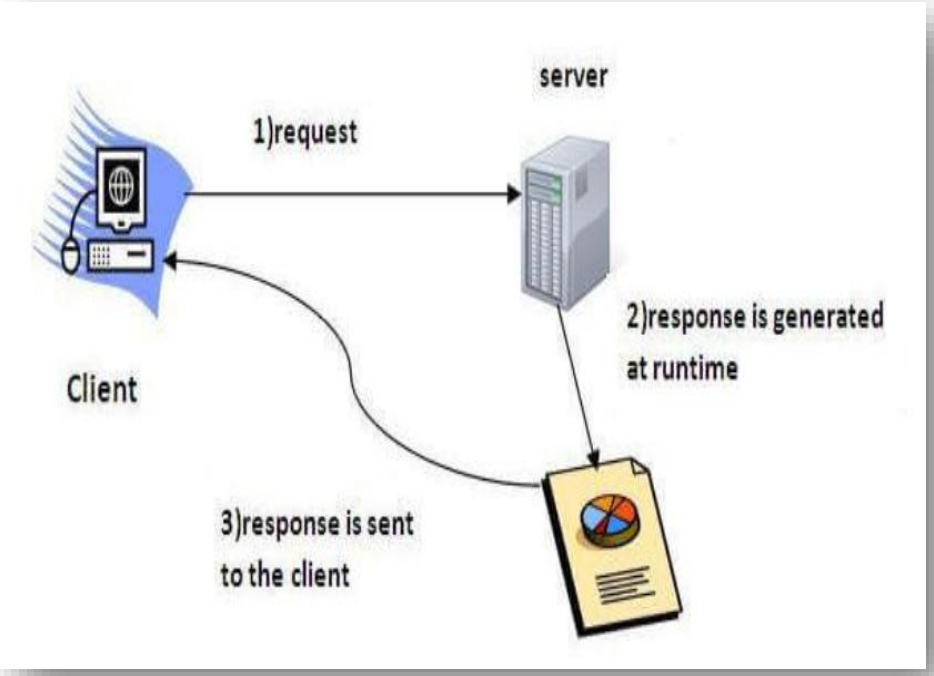
- **Processing user input**
- **Database connectivity**
- **Dynamic content generation**
- **Business logic on the server**

Web Application:

What is a Web Application?

A Web Application is an application that:

- **Runs on a server**
- **Is accessed through a web browser**
- **Requires an internet or network connection**
- **Example : GMAIL etc.**



Server : It is a computer which provides service to other computers.

Server accepts then request sent by the client , processes it , generates the response , and sends back response to the client.

Client : A Client is a system or software that:

Sends requests to the server

Receives responses from the server

Servlet API

- The javax.servlet and javax.servlet.http packages represent interfaces and classes for servlet api.

- The javax.servlet package contains many interfaces and classes (like , Servlet interface , GenericServet class etc) that are used by the servlet or web container. These are not specific to any protocol.

- The javax.servlet.http package contains interfaces and classes(like, HttpServlet class etc) that are responsible for http requests only.

WAYS TO CREATE A SERVLET

- **Create a class :**

1. **by implementing Servlet interface**
2. **by extending GenericServlet class.**
3. **by extending HttpServlet class.**

SERVLET INTERFACE

- **Servlet interface provides common behavior to all the servlets**

.

- **Servlet interface defines methods that all servlets must implement.**

- **Servlet interface needs to be implemented for creating any servlet (either directly or indirectly).**

- **It provides 3 life cycle methods that are used to initialize the servlet, to service the requests, and to destroy the servlet and 2 non-life cycle methods.**

Methods of Servlet interface

There are 5 methods in Servlet interface :

1. `public void init(ServletConfig config)` : **initializes the servlet. It is the life cycle method of servlet and invoked by the web container only once.**
2. `public void service(ServletRequest request,ServletResponse response)` : **accepts the request and provides response for the incoming request. It is invoked at each request by the web container.**
3. `public void destroy()` : **is invoked only once and indicates that servlet is being destroyed.**
4. `public ServletConfig getServletConfig()` : **returns the object of ServletConfig.**
5. `public String getServletInfo()` : **returns information about servlet such as writer, copyright, version etc.**

GenericServlet class

- **GenericServlet class implements Servlet interface. It provides the implementation of all the methods of these interfaces except the service method.**

- **GenericServlet class can handle any type of request so it is protocol-independent.**

- **We may create a generic servlet by inheriting the GenericServlet class and providing the implementation of the service method.**

Service() method to be overridden is :

- public void service(ServletRequest request,ServletResponse response) : **accepts the request and provides response for the incoming request. It is invoked at each request by the web container.**

- **The service() method accepts two parameters :**
- **ServletRequest (interface) : An object of ServletRequest is used to provide the client request information to a servlet such as parameter names and values, attributes etc.**
  - It consists of a method :
    - **public String getParameter(String name) : it is used to obtain the value of a parameter by name.**
  - **Etc...**

- **ServletResponse (interface) : An object of ServletRequest is used to format and send the response back to the client.**
  - It consists of methods :
    - **void setContentType(String) : Sets the type of the response data. e.g. text, html etc.**
    - **java.io.PrintWriter getWriter() : Returns the PrintWriter object.**
  - **Etc..**

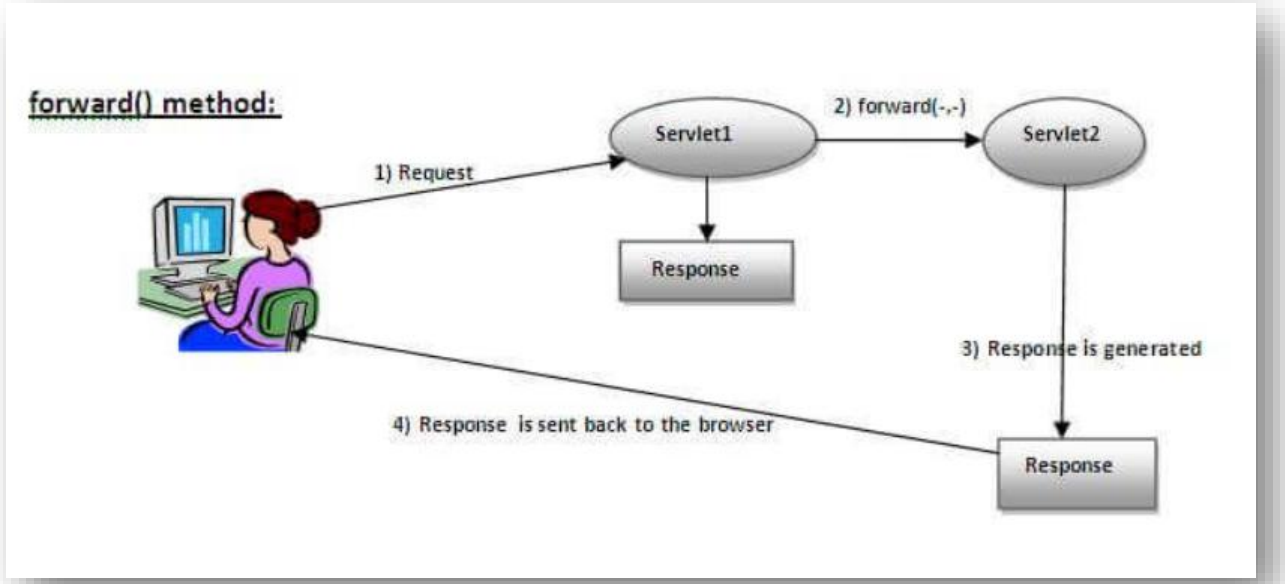
REQUEST DISPATCHER

- **The RequestDispatcher interface provides the facility of dispatching the request(forwarding) to another resource it may be html, servlet or jsp.**
- **This interface can also be used to include the content of another resource also.**

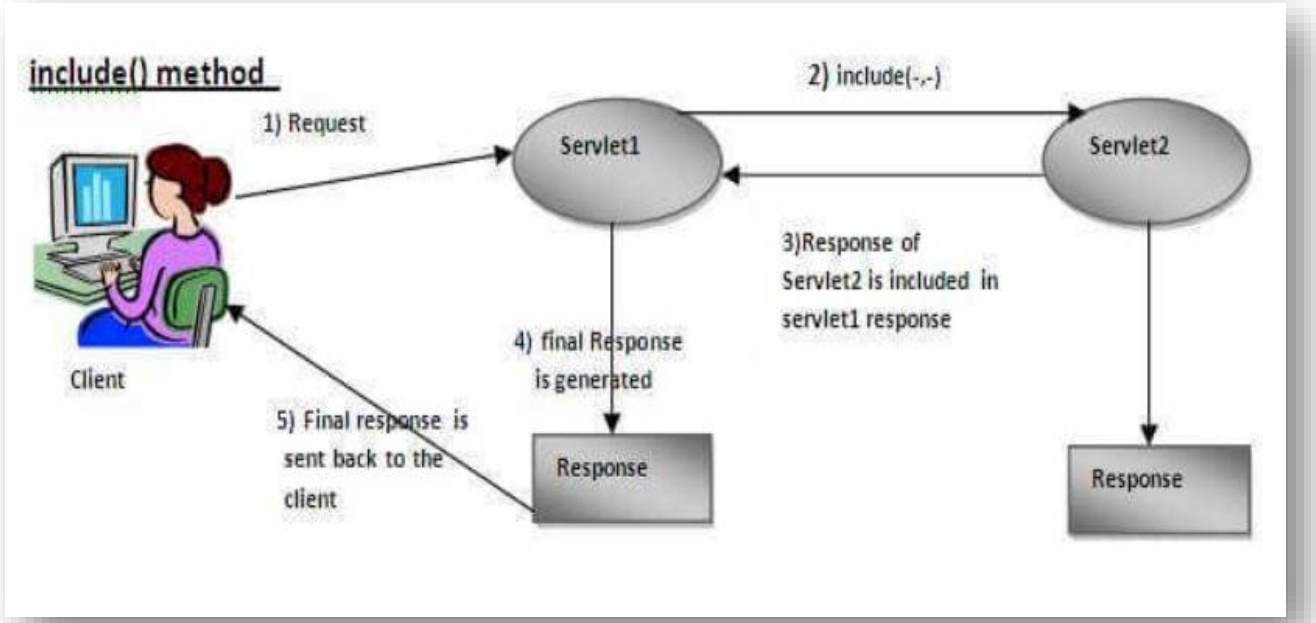
Methods of RequestDispatcher interface

- **public void forward(ServletRequest request,ServletResponse response) : Forwards a request from a servlet to another resource (servlet, JSP file, or HTML file) on the server.**

- **public void include(ServletRequest request,ServletResponse response) :Includes the content of a resource (servlet, JSP page, or HTML file) in the response.**



Response of second servlet is sent to the client. Response of the first servlet is not displayed to the user. It is forwarded to the second servlet.



Response of second servlet is included in the response of the first servlet that is being sent to the client.

How to get the object of RequestDispatcher

The getRequestDispatcher() method of ServletRequest interface returns the object of RequestDispatcher.

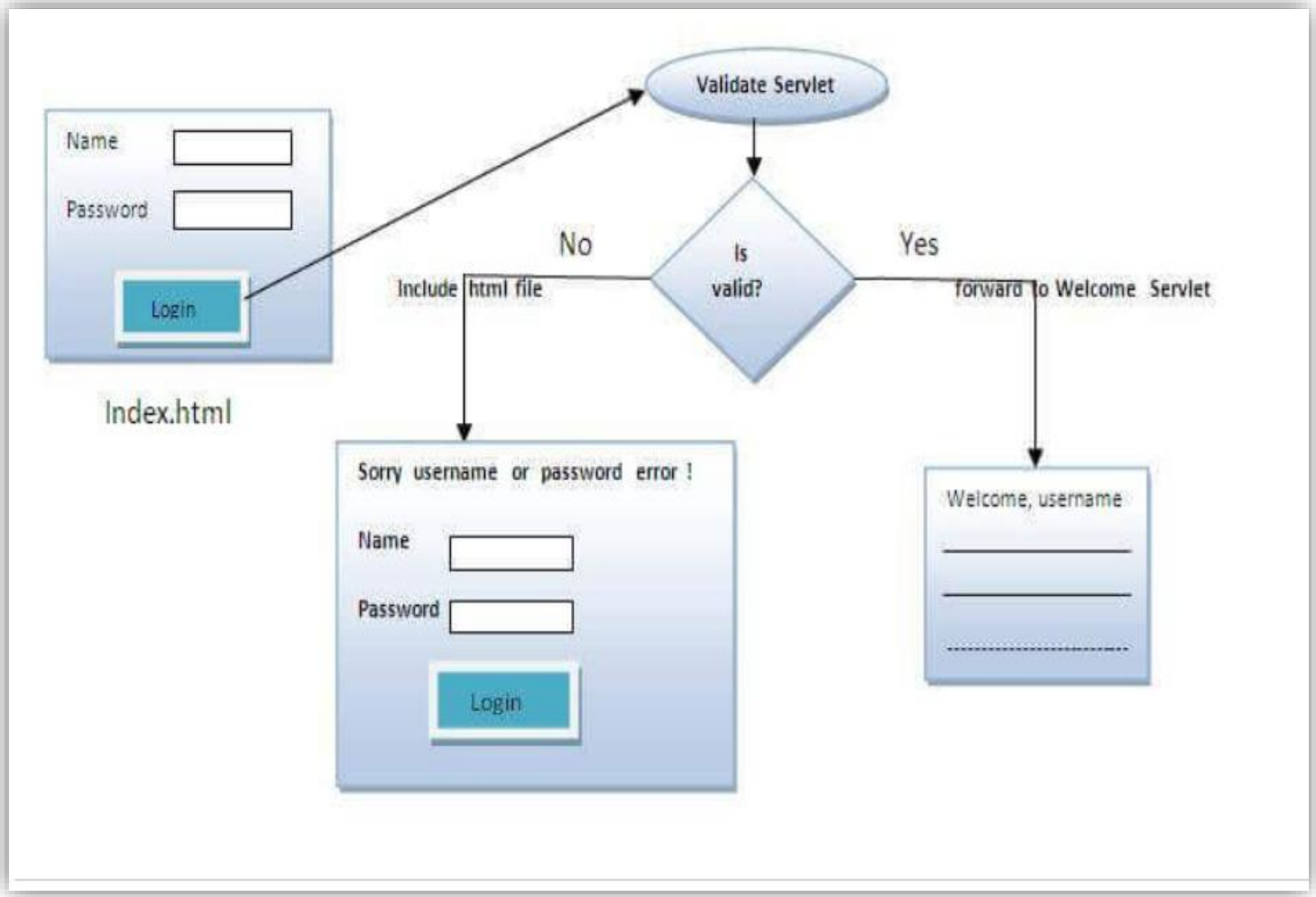
Syntax :

```
public RequestDispatcher getRequestDispatcher(String resource);
```

Example :

```
RequestDispatcher rd=request.getRequestDispatcher(resource);
//resource can be url-pattern of the servlet  , jsp or html file name.
```

Example :



In this example, we are validating the password entered by the user. If password is correct as per the condition, it will forward the same request to the WelcomeServlet, otherwise it will include an error message : " sorry username or password error! " in login html file.

HttpServlet class

- **The HttpServlet class extends the GenericServlet class.**
- **It has all the properties inherited from GenericServlet class.**
- **In HttpServet class , service() method is also overridden.**
- **Along with the inherited methods , It provides http specific methods such as doGet, doPost, doHead, doTrace etc to handle specific type of requests.**

THE METHODS TO HANDLE DIFFERENT REQUESTS ARE :

- protected void doGet(HttpServletRequest req, HttpServletResponse res) **handles the GET request. It is invoked by the web container.**
- protected void doPost(HttpServletRequest req, HttpServletResponse res) **handles the POST request. It is invoked by the web container.**
- protected void doPut(HttpServletRequest req, HttpServletResponse res) **handles the PUT request. It is invoked by the web container.**
- protected void doDelete(HttpServletRequest req, HttpServletResponse res) **handles the DELETE request. It is invoked by the web container.**
- **Etc...**

These methods accepts two parameters :

- **HttpServletRequest (interface) : Extends the ServletRequest interface to provide request information for HTTP servlets.**
- **An object of HttpServletRequest is used to provide the client request information to a Http servlet such as parameter names and values, attributes etc.**

  It inherits the methods of ServletRequest interface such as :

  - **public String getParameter(String name)**
  - **Etc...**

- **HttpServletResponse (interface) : Extends the ServletResponse interface to provide HTTP-specific functionality in sending a response.**
- **An object of HttpServletResponse is used to format and send the response back to the client from HTTP servlets.**

  It inherits the methods of ServletResponse interface such as :

  - **void setContentType(String) : Sets the type of the response data. e.g. text, html etc.**
  - **java.io.PrintWriter getWriter() : Returns the PrintWriter object.**
  - **Etc..**

sendRedirect() method

- **It is a method defined in HttpServletResponse interface in java.**
- **It is used to redirect the client to a different URL.**

Syntax : void sendRedirect(String location) throws IOException

- **Parameters:**

  location: The URL to which the client is redirected. It can be an absolute or relative URL.

It can be used for navigating within the same server as well as outside the server.

NOTE :

When to Use:

1. **forward() : When transferring control to another resource within the same application and When you want to share request data with the target resource.**

   → Supported by GenericServlet class and HttpServlet class.

2. **sendRedirect() : When redirecting to an external resource or a different application running on different server.**

   → Supported only in HttpServlet class.

PRINTWRITER

- The Java PrintWriter class implements the Writer class. It is used to print the formatted representation of objects to the text output stream.

- It provides convenient methods for writing various forms of data such as integer, double, and String or Object in the form of text the browser , in to a file or any other output destination.

- It consists of print() method and write() methods for printing / displaying data .

- In servlets , we get the PrintWriter object using getWriter() of ServlerResponse interface.

  Ex : PrintWriter ref = response.getWriter();

Servlets
Sample Programs

NOTE

- All the Servlets(Java classes) should be created inside src/main/java folder.
- All html , css files should be created inside Webapp folder.

1 Creating Servlet By Extending GenericServlet Class.

First.html

```
<body>

<form action="demo">

NAME : <input type="text" name="myname">

<input type="submit">

</form>

</body>
```

FirstServlet.java

```java
// 3. specify the URL for accessing this servlet
@WebServlet("/demo")
//1. create a class by extending GenericServlet class
public class FirstServlet extends GenericServlet
{
    // 2. override service()
    @Override
    public void service(ServletRequest req, ServletResponse res)
                                throws ServletException, IOException
    {

        // fetch the data sent by the client
        String name =   req.getParameter("myname");

        // creating PrintWriter object for generating response of this servlet
        PrintWriter pw = res.getWriter();

        pw.print("<h1>WELCOME TO MY FIRST SERVLET PROGRAM..."+name+"</h1>");

    }
}
```

2

Details.html

```
<body>
<h2>PERSONAL DETAILS FORM</h2>

<form action="mydetails">

NAME : <input type="text" name="myname"><br><br>
AGE : <input type="number" name="myage"><br><br>
PHONE NUMBER : <input type="number" name="myphone"><br><br>
ADDRESS : <input type="text" name="myaddress"><br><br>
DATE OF BIRTH : <input type="date" name="mydate"><br><br>

<input type="submit">

</form>
</body>
```

DetailsServlet.java

```java
@WebServlet("/mydetails")
public class DetailsServlet extends GenericServlet
{
    @Override
    public void service(ServletRequest req, ServletResponse res)
                        throws ServletException, IOException
    {

        String name = req.getParameter("myname");
        String age = req.getParameter("myage");
        String phone  = req.getParameter("myphone");
        String address = req.getParameter("myaddress");
        String dob = req.getParameter("mydate");

        PrintWriter pw = res.getWriter();
        pw.print("<html><body>");
        pw.print("<h1>YOUR DETAILS</h1>");
        pw.print("<br><br><b>NAME : </b>"+name);
        pw.print("<br><br><b>AGE : </b>"+age);
        pw.print("<br><br><b>ADDRESS : </b>"+address);
        pw.print("<br><br><b>PHONE NUMBER : </b>"+phone);
        pw.print("<br><br><b>DATE OF BIRTH : </b>"+dob);
        pw.print("</body></html>");
    }

}
```

3 Use of RequestDispatcher interface.

Login.html

```
<body>
<h1>LOGIN HERE...</h1>

<form action="login">

USERNAME : <input type="text" name="uname"><br><br>
PASSWORD : <input type="text" name="password"><br><br>
<input type="submit" value="LOGIN">

</form>

</body>
```

LoginServlet.java

```java
@WebServlet("/login")
public class LoginServlet extends GenericServlet {

    @Override
    public void service(ServletRequest req, ServletResponse res)
                                    throws ServletException, IOException
    {
        String username = req.getParameter("uname");
        String password = req.getParameter("password");

        // validation --> username = suresh and password = 12345
        if(username.equals("suresh") && password.equals("12345"))
        {
            RequestDispatcher rd = req.getRequestDispatcher("display");
            rd.forward(req, res);
        }
        else
        {
            res.setContentType("text/html");

            PrintWriter pw = res.getWriter();
            pw.print("INCORRECT USERNAME OR PASSWORD...!");
            RequestDispatcher rd = req.getRequestDispatcher("login.html");
            rd.include(req, res);
        }
    }


}
```

DisplayServlet.java

```java
@WebServlet("/display")
public class DisplayServlet extends GenericServlet{

    @Override
    public void service(ServletRequest req, ServletResponse res)
                                throws ServletException, IOException
    {

        String username = req.getParameter("uname");
        PrintWriter pw = res.getWriter();
        pw.print("<h1>WELCOME ..."+username+"</h1>");
    }
}
```

4 Creating Servlet By Extending HttpServlet Class.

- **Get request is sent by the client which is handled by servlet using doGet() method.**

sample.html

```
<body>
<form action="sample">

NAME : <input type="text" name="myname"><br><br>
<input type="submit">

</form>
```

SampleServlet.java

```java
@WebServlet("/sample")
public class SampleServlet extends HttpServlet
{
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
                                                    throws ServletException, IOException
    {

            String name = req.getParameter("myname");
            PrintWriter pw = resp.getWriter();
            pw.print("Welcome..."+name);
    }
}
```

5 Creating Servlet By Extending HttpServlet Class.

- **Post request is sent by the client which is handled by servlet using doPost() method.**

test.html

```
<body>
<form action="test" method="post">

NAME : <input type="text" name="myname"><br><br>
AGE : <input type="number" name="myage"><br><br>
PHONE NUMBER : <input type="number" name="myphone"><br><br>
ADDRESS : <input type="text" name="myaddress"><br><br>
DATE OF BIRTH : <input type="date" name="mydate"><br><br>
<input type="submit">

</form>
</body>
</html>
```

TestServlet.java

```java
@WebServlet("/test")
public class TestServlet extends HttpServlet
{
    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {

        String name = req.getParameter("myname");
        String age = req.getParameter("myage");
        String phone  = req.getParameter("myphone");
        String address = req.getParameter("myaddress");
        String dob = req.getParameter("mydate");

        PrintWriter pw = resp.getWriter();
        pw.print("<html><body>");
        pw.print("<h1>YOUR DETAILS</h1>");
        pw.print("<br><br><b>NAME : </b>"+name);
        pw.print("<br><br><b>AGE : </b>"+age);
        pw.print("<br><br><b>ADDRESS : </b>"+address);
        pw.print("<br><br><b>PHONE NUMBER : </b>"+phone);
        pw.print("<br><br><b>DATE OF BIRTH : </b>"+dob);
        pw.print("</body></html>");

    }
}
```

6 Use of sendRedirect() method.

search.html

```
<body>

<form action="search">

Search : <input type="text" name="name">
<input type="submit">

</form>
</body>
</html>
```

SearchServlet.java

```java
@WebServlet("/search")
public class SearchServlet extends HttpServlet
{
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {

        String data = req.getParameter("name");
        if(data.equalsIgnoreCase("java"))
        {
            // redirecting response to java tutorial of javatpoint
            resp.sendRedirect("https://www.javatpoint.com/java-tutorial");
        }
        else
        {
            // redirecting response to google home page
            resp.sendRedirect("https://www.google.com/");
        }
    }
}
```